

TP 1: Introduction to Data Analysis with Python

Required Libraries

NumPy — numerical computation; Pandas — data manipulation; Matplotlib — visualization; and Seaborn — statistical visualization.

Exercise 01

Create a small student dataset and name it `df = pd.DataFrame(data)`

```
1 data = {      "Student": ["S1", "S2", "S3", "S4", "S5",  
2                  "S6", "S7", "S8", "S9", "S10"],  
3      "Age": [21, 22, 20, 24, 21, 23, 22, 20, 25, 21],  
4      "Gender": ["F", "M", "F", "M", "F", "M", "F", "M", "M", "F"],  
5      "Study_Hours": [12, 8, 15, 5, 10, 7, 14, 6, 4, 11],  
6      "Grade": [15.5, 12.0, 17.0, 8.5, 13.5, 10.0, 16.0, 9.0, 7.5,  
7              14.0],  
      "Absences": [2, 5, 1, 14, 4, 9, 2, 11, 18, 3]}
```

1. Display the first five observations and explain the roll of `df.head()`.
2. Display the last five observations with `df.tail()`.
3. Determine the dimensions of the dataset with `df.shape`.
4. Examine the Data Types `df.info()`
5. Descriptive Statistics with `df.describe()` and provide: count; mean; standard deviation; minimum; quartiles; and maximum, For example:

(a) Calculate the mean grade:

```
1 mean_grade = df["Grade"].mean()  
2 print("Mean grade:", mean_grade)
```

(b) Calculate the median grade:

```
1 median_grade = df["Grade"].median()  
2 print("Median grade:", median_grade)
```

(c) Calculate the median number of absences:

```
1 df["Absences"].median()
```

(d) Minimum and Maximum

```
1 print("Minimum grade:", df["Grade"].min())  
2 print("Maximum grade:", df["Grade"].max())
```

(e) minimum absences and the maximum absences

6. Calculate the Standard Deviation with

```
1 std_grade = df["Grade"].std()
2 print("Standard deviation:", std_grade)
```

7. What does a large standard deviation indicate?

8. Frequency Analysis

(a) For categorical variables:

```
1 df["Gender"].value_counts()
```

(b) To calculate proportions:

```
1 df["Gender"].value_counts(normalize=True)
```

9. Histogram

```
1 plt.figure(figsize=(8, 5))
2 plt.hist(df["Grade"], bins=5)
3 plt.xlabel("Grade")
4 plt.ylabel("Frequency")
5 plt.title("Distribution of Student Grades")
6 plt.show()
```

(a) Is the distribution approximately symmetric?

(b) Which grade range contains the largest number of students?

(c) Are there potentially unusual values?

10. Boxplot: The boxplot allows to identify: median; quartiles; dispersion; and potential outliers.

```
1 plt.figure(figsize=(8, 5))
2 sns.boxplot(x=df["Grade"])
3 plt.xlabel("Grade")
4 plt.title("Boxplot of Student Grades")
5 plt.show()
```

11. Detecting Outliers with IQR

```
1 Q1 = df["Grade"].quantile(0.25)
2 Q3 = df["Grade"].quantile(0.75)
3 IQR = Q3 - Q1
4 print("Q1 =", Q1)
5 print("Q3 =", Q3)
6 print("IQR =", IQR)
```

(a) Calculate the limits:

```
1 lower_bound = Q1 - 1.5 * IQR
2 upper_bound = Q3 + 1.5 * IQR
3 print("Lower bound:", lower_bound)
4 print("Upper bound:", upper_bound)
```

(b) Identify possible outliers:

```
1 outliers = df[
2     (df["Grade"] < lower_bound) |
3     (df["Grade"] > upper_bound)]
4 print(outliers)
```

12. Missing Values

```
1 df.loc[2, "Grade"] = np.nan
```

(a) Check the dataset:

```
1 print(df)
```

(b) Calculate missing values:

```
1 df.isnull().sum()
```

(c) Which variable contains a missing value?

13. Handling Missing Values

(a) Calculate the mean:

```
1 mean_grade = df["Grade"].mean()
```

```
1 print(mean_grade)
```

(b) Replace the missing value:

```
1 df["Grade"] = df["Grade"].fillna(mean_grade)
```

(c) Verify:

```
1 df.isnull().sum()
```

14. Duplicate Data:

(a) To check duplicates:

```
1 df.duplicated().sum()
```

(b) To display duplicated rows:

```
1 df[df.duplicated()]
```

- (c) To remove duplicates:

```
1 df = df.drop_duplicates()
```

15. Scatter Plot: We investigate the relationship between study hours and grade.

```
1 plt.figure(figsize=(8, 5))
2 sns.scatterplot(
3     data=df,
4     x="Study_Hours",
5     y="Grade")
6 plt.xlabel("Study Hours")
7 plt.ylabel("Grade")
8 plt.title("Study Hours vs Grade")
9 plt.show()
```

- (a) Does grade tend to increase with study hours?
(b) Are there unusual observations?
(c) Does the relationship appear linear?

16. Correlation Matrix

- (a) Calculate correlations:

```
1 correlation_matrix = df[["Age", "Study_Hours", "Grade", "
    Absences"]].corr()
2 print(correlation_matrix)
```

- (b) Visualize them:

```
1 plt.figure(figsize=(8, 6))
2 sns.heatmap(correlation_matrix,
3     annot=True,
4     cmap="coolwarm",
5     fmt=".2f")
6 plt.title("Correlation Matrix")
7 plt.show()
```

- (c) Identify which variables have:
- strong positive correlation;
 - strong negative correlation; and weak correlation.

17. Group Analysis

- (a) Calculate the average grade by gender:

```
1 df.groupby("Gender")["Grade"].mean()
```

- (b) Calculate average study hours:

```
1 df.groupby("Gender")["Study_Hours"].mean()
```

- (c) Calculate average absences:

```
1 df.groupby("Gender")["Absences"].mean()
```